

SOS

অতি সংক্ষিপ্ত প্রশ্নোত্তর :

***** ১। ইউনিট টেস্টিং (Unit Testing) কী?**

বাকাশিবো- ২০২৪

উত্তরঃ একটি Software code এর Unit হলো Function বা Method. এই ইউনিটকে টেস্ট করার প্রক্রিয়াকে ইউনিট টেস্টিং বলে। এই টেস্ট কোডগুলো লেখার জন্য বিভিন্ন ধরনের Frame work, Tools ইত্যাদি ব্যবহার করা হয়। যেমন : Data flow Testing, Control flow Testing, Branch Coverage Testing, Statement Coverage Testing, Decision Coverage Testing ইত্যাদি।

[**Note:** Framework: Software তৈরিতে Framework হলো একটি সাহায্যকারী কাঠামো, যার উপর ভিত্তি করে অন্য Software তৈরি ও উন্নয়ন করা যায়। Software উন্নয়নকে সুবিধাদানের জন্যই মূলত তৈরি করা হয়।]

*** ২। ইউনিট টেস্টিং এর প্রয়োজনীয়তা কী?**

বাকাশিবো- ২০২৩

উত্তরঃ Manual Testing এর ক্ষেত্রে বিশাল বড় বড় কোড বা বিভিন্ন ধরনের Function সমূহকে আলাদা আলাদাভাবে টেস্ট করা অনেকটা কষ্টসাধ্য এবং সময়সাপেক্ষ ব্যাপার। এই অসুবিধা দূর করে দক্ষতার সাথে



কোডসমূহকে টেস্ট করার জন্য ইউনিট টেস্টিং খুবই কার্যকরী একটি উপায়। এই ক্ষেত্রে বিভিন্ন Framework, Tools ইত্যাদি ব্যবহৃত হয়।

**** ৩। পাইথন বিল্ট-ইন ইউনিট টেস্ট ইউনিটগুলো নাম লেখ।**

উত্তরঃ পাইথন বিল্ট-ইন ইউনিট টেস্ট ইউনিটগুলো হলো : unittest, unittest2, mock, nose, tox ইত্যাদি।

*** ৪। বহুল ব্যবহৃত Python module গুলোর নাম লেখ।**

উত্তরঃ unittest, DocTest, Pytest, Robot, Nose ইত্যাদি।

**** ৫। Unittest framework ব্যবহার করে Testing এর কী কী ফলাফল পাওয়া যায়?**

উত্তরঃ i) Ok: যদি সকল test pass করে, তবে Ok Return করে।

ii) Failure: যদি কোনো Test fail করে, তবে Assertion Error Exception Raise করে।

iii) Error: Assertion Error এর পরিবর্তে অন্য কোনো Error সংঘটিত হলে।

SOS**সংক্ষিপ্ত প্রশ্নোত্তর :**

*** ১। পাইথনে ইউনিট টেস্টিং এর গুরুত্ব সংক্ষেপে লেখ।

উত্তরঃ পাইথনে ইউনিট টেস্টিং এর গুরুত্ব :

- i) কোড সেকশনকে আলাদা করা।
- ii) প্রতিটি কোড সঠিক কিনা, তা যাচাই করা।
- iii) কোডের সঠিকতা আলাদা আলাদাভাবে যাচাই করা।
- iv) প্রতিটি মেথডকে আলাদা আলাদাভাবে পরিক্ষা করা।
- v) Develop cycle এর প্রথম দিকে Bug গুলো সঠিক করতে।
- vi) কোড দ্রুত পরিবর্তন করতে সক্ষম করতে।
- vii) Code পুনরায় ব্যবহার করতে সাহায্য করে।
- viii) অতি কম সময়ের মধ্যে Bug খুঁজে বের করা যায়।
- ix) URL (Uniform Resource Locator) সংস্করণ করতে হবে কিনা তা সাথে সাথেই বুঝা যায়।

** ২। ইউনিট টেস্টিং এর সুবিধাগুলো কী কী?

উত্তরঃ i) ফাংশনটি কীভাবে কাজ করে তা যে কেউ কোড দেখার আগেই Unit test দেখে ধারণা করতে পারবে।

ii) Unit test এর case গুলো দেখলে বুঝা যায় কোনো বিশেষ case মিস হয়েছে কিনা, যদি মিস হয় তবে সেই Case যুক্ত করে test Run করলে সেই case এর জন্য ফাংশনটি ঠিকভাবে Output দিচ্ছে কিনা।



iii) কেউ যদি ফাংশনে কোনো পরিবর্তন করে তাহলে অনেক সময় ফাংশনে অনাকাঙ্ক্ষিত bug আসতে পারে। সেক্ষেত্রে যদি ফাংশনের Unit test ঠিকমত লেখা থাকে, তবে bug না আসার সম্ভাবনা থাকে। কারণ bug ঢুকলে Unit test fail করবে।

SOS**রচনামূলক প্রশ্নোত্তর :**

*** ১। Unit testing এর Structure ব্যাখ্যা কর। বাকশিবো- ২০২৩

উত্তরঃ একটি Software code এর Unit হলো Function বা Method. এই ইউনিটকে টেস্ট করার প্রক্রিয়াকে ইউনিট টেস্টিং বলে।

Basic test structure : Unit test কে দু'ভাবে বর্ণনা করা যায়। যথা :

১) কোড ব্যবহার করে ২) নিজেই test করে।

```
import unittest
class TestingSum(unittest.TestCase):
    def test_sum(self):
        self.assertEqual(sum([2, 3, 5]), 10, "It should be 10")
    def test_sum_tuple(self):
        self.assertEqual(sum((1, 3, 5)), 10, "It should be 10")
if __name__ == '__main__':
```



`unittest.main()`

Unit testing এর সম্ভাব্য ফলাফল তিন ধরনের হতে পারে।

যথা :

i) **Ok:** যদি সকল test pass করে, তবে Ok Return করে।

ii) **Failure:** যদি কোন Test fail করে, তবে Assertion Error Exception Raise করে।

iii) **Error:** Assertion Error এর পরিবর্তে অন্য কোনো Error সংঘটিত হলে।

**** ২।** পাইথন ইউনিট টেস্ট এর রিকোয়ারমেন্টস বর্ণনা কর।

উত্তরঃ Unit test: Code test করার জন্য প্রয়োজনীয় সকল tools, unit test এ বিদ্যমান। এটি পাইথন এর Default unit testing framework, যা Standard library এর অন্তর্ভুক্ত। Unit test ব্যবহার করে কীভাবে প্রোগ্রাম লেখা, টেস্ট ও নির্বাহ করা হয় তা নিম্নে দেখানো হলো :

```
def add(x,y):  
    return x+y  
def check_even(num):  
    if(num%2==0):  
        return True  
    else:
```

```
return False
```

উপরের ফাংশন দুইটি টেস্ট করার জন্য একই Directory তে test code লিখতে হবে।

```
import unittest
from test import add, check_even
class MyTest(unittest.TestCase):
    def test_add(self):
        self.assertEqual(add(4,5),9)
    def test_check_even(self):
```

উপরের প্রোগ্রামে add () ফাংশন দুটি সংখ্যা যোগ করে। তাই test করে দেখেছি যে add () ফাংশনে 4, 5 pass করলে Return কৃত মান 9 এর সমান হয় কিনা।

check_even() ফাংশনটি চেক করে দেখে যে, কোনো সংখ্যা জোড় কিনা? জোড় হলে True Return করবে, বিজোড় হলে False Return করবে। check_even() ফাংশনে 4 কে pass করে দেখেছি তা True Return করে কিনা। এই পদ্ধতি ব্যবহার করে, ফাংশনের কোড test করা যায়।

